

DSBA 6171 · HEALTHTECH DOMAIN PROJECT

Assignment 2

Structured Data for AI: Meaning, Context & Trust

HealthTech Medical Claims Review Microcosm

Team GitHub Repository

github.com/Freh-dev/dsba6171-healthtech-domain-project

Domain: Healthcare Payer Claims Adjudication

DSBA 6171 — Data Engineering for AI

Team Members

Elijah · Frehiwot · Daniel · Dzmitry · Ketan

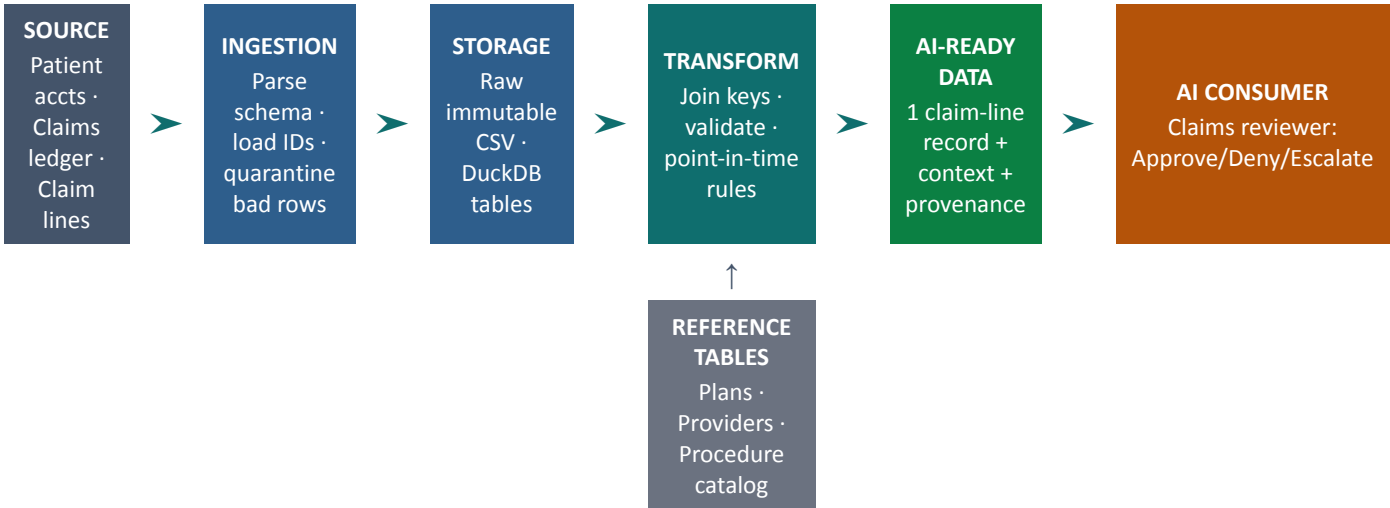
1. Structured Data Architecture & AI Consumption

Decision the AI supports: *for this member, plan, provider, procedure, and date of service - should each claim line be approved, denied, or escalated for human review?*

Three datasets form the decision spine - patient_accounts → claims_ledger → claim_line_items - supported by reference tables (insurance_plans, providers, procedure_catalog) that resolve the IDs the spine depends on. The AI decision is kept at claim-line grain since one claim can bundle several procedures across different dates, amounts, and outcomes.

Dataset	Type	Grain (1 row =)	Keys & relationships	AI consumption	Key assumption	Access/storage pattern
patient_accounts.csv	Entity / enrollment master	One member's enrollment record & coverage interval (e.g., M-1001, PLAN-001, eff. 2025-01-01)	PK member_id · FK plan_id → insurance_plans · FK primary_provider_id → providers	Current case context — was the member covered on the line's service date?	effective/termination dates accurately reflect coverage; else an ineligible claim is evaluated	Batch CSV → DuckDB table; validate schema, keys, and enrollment dates.
claims_ledger.csv	Event / transaction header	One submitted claim: receipt date, service interval, status, diagnoses, claim-level totals	PK claim_id · FKs member_id, provider_id, plan_id · 1 claim → many claim_line_items	Claim & historical context — the container (status, diagnoses, totals) around the line decision	Claim totals reconcile to sum of lines; else payment/utilization features are overstated	Batch CSV → DuckDB table; validate schema, keys, and claim totals vs. lines.
claim_line_items.csv	Event / transaction detail	One billed service line within a claim (e.g., LI-10001: CLM-10001, CPT-99213, 2026-08-01)	PK claim_line_id · FK claim_id · FK procedure_code → procedure_catalog · line_number = order	Primary feature/input & decision grain — procedure, dates, amounts, denial code drive approve/deny/escalate	service_date is correct, so point-in-time coverage & policy are correctly selected	Batch CSV → DuckDB table; validate schema, keys, dates, and relationships.

Reference tables (insurance_plans, providers, procedure_catalog) resolve the IDs used across the spine and supply policy context, e.g. procedure_catalog.coverage_policy_id → the rule governing a procedure. **Ingestion approach:** batch-load source CSVs, parse against the declared schema, preserve member_id/claim_id/claim_line_id, validate required fields, and quarantine malformed or near-duplicate rows before they reach storage. **Storage:** raw CSVs load into an immutable evidence layer; queryable copies load into DuckDB with PK/FK checks; tables are joined at query time so every AI-ready record traces back to its source rows.



Required diagram — Source → Ingestion → Storage → Transformation → AI-Ready Structured Data → AI Consumer

Result: the AI receives structured data with the correct **meaning** (declared grain), **context** (member, claim, plan, provider, procedure), **timing** (service date and coverage/policy validity), and **traceability** (source keys and quality flags) — supporting a human claims reviewer without replacing adjudication authority.

2. Data Modeling & AI Context Integrity

Scenario: *One-to-many join duplication*

Data Engineering Issue. The AI reviews individual service lines using claim-level context. In [claims ledger](#), one row represents a claim; in [claim_line_items](#), one row represents a service line. This is their grain. Joining on `claim_id` repeats the claim total for every service line. The JOIN is valid, but summing that repeated total overstates the billed amount.

What AI Receives. Claim *CLM-10058* has a total billed amount of \$1,255.72 and three service lines. The incorrect calculation adds the total three times, so AI receives \$3,767.16.

Possible AI Interpretation. Asked to summarize the claim, AI could report: “*The total billed amount is \$3,767.16.*” This reasonably reflects its input but misrepresents the actual claim.

Business Consequence. A reviewer receives an overstated cost summary and may initiate an unnecessary investigation. This can waste review time, delay processing, and distort claims reporting.

Architectural Control. Preserve the original claim total while retaining service-line detail. Sum service-line amounts separately and compare them with the original total before supplying data to AI. Flag mismatches for review. Both correct totals for this claim are \$1,255.72.

Supporting evidence: [Jupyter Notebook](#) demonstrates the source rows, JOIN, and both calculations.

3. Schema, Semantic Contract & AI Context Quality

A. Schema Expectations

Anchor dataset: `claim_line_items.csv` — the decision-grain table the AI reasons over line-by-line (per Section 1).

Schema/version identifier: `claim_line_items_schema_v1.0`

Field	Data Type	Nullable	Allowed Values / Constraints	Key Status
-------	-----------	----------	------------------------------	------------

claim_line_id	string (LI-#####)	No	Unique	PK
claim_id	string (CLM-#####)	No	Must exist in claims_ledger.claim_id	FK
line_number	integer	No	≥ 1 ; unique within claim_id (a claim can have multiple lines, e.g. CLM-10097 has lines 1–2)	—
procedure_code	string (CPT-#####)	No	Must exist in procedure_catalog.procedure_code	FK
service_date	date (ISO 8601 YYYY-MM-DD)	No	Must fall within the parent claim's service window	—
units	integer	No	≥ 1 (observed values: 1, 2)	—
billed_amount	decimal(10,2)	No	> 0	—
allowed_amount	decimal(10,2)	No, but 0.00 is valid (not-yet-adjusted)	≥ 0 ; $\leq$ billed_amount when > 0	—
paid_amount	decimal(10,2)	No, but 0.00 is valid (not-yet-paid)	≥ 0 ; $\leq$ allowed_amount when > 0	—
modifier_code	string	Yes	Valid CPT modifier (observed: RT, LT, 50, 26) or blank	—
denial_reason_code	string	Yes	Populated only when the line is denied/adjusted; pattern <GroupCode>-<ReasonNumber> (observed: CO-10, CO-11, CO-12, CO-13, CO-50, OA-23, PR-16, PR-6)	—

Required Semantic-Failure Example

The semantic rule: billed_amount is defined by the contract as the total USD amount billed for the entire service line, already reflecting units. It is not a per-unit amount.

Schema-valid but semantically wrong record: use claim_line_id = LI-10031 (claim_id = CLM-10021, procedure CPT-29822, units = 2) as a controlled failure. Assume an upstream source changes billed_amount to a per-unit amount but keeps the same column name and numeric type. A value of billed_amount = 3338.03 still passes the schema checks because it is a positive decimal, yet it violates the semantic contract because the field no longer means total line amount.

What AI receives: units = 2 and billed_amount = 3338.03, with no schema error indicating that the unit semantics changed.

Possible AI interpretation: because the contract says billed_amount is the line total, the AI can reasonably report \$3,338.03 as the total billed amount. If 3338.03 is actually a per-unit amount, the true line total is \$6,676.06. The AI is following the declared schema while receiving semantically incorrect business meaning.

Business consequence: the line cost is understated by 50%, which can distort claim-level and portfolio-level cost/utilization summaries and can suppress review signals that depend on billed amount. The failure is caused by a semantic unit change, not by an invalid data type or join.

Corrected / canonical representation: normalize the source semantics before the AI-ready layer.

claim_line_id	claim_id	procedure_code	units	unit_billed_amount	total_billed_amount
LI-10031	CLM-10021	CPT-29822	2	\$3,338.03	\$6,676.06

Supporting evidence: metadata/schema_claim_line_items_v1.csv defines the technical structure; data/structured/raw/claim_line_items.csv provides the source row shape. The per-unit change above is an intentional controlled semantic-failure scenario used to test the contract, not a claim that the raw source file currently contains that defect.

B. Semantic Contract & AI Context Quality

Business definition & grain: claim_line_items.csv is the decision-grain dataset for billed services. One row represents one service line within one claim. claim_line_id is the row identifier, and (claim_id, line_number) must remain unique. Enrichment with claim, member, plan, provider, procedure, or policy context must not duplicate or change that grain.

Units, null/zero semantics & point-in-time meaning: billed_amount, allowed_amount, and paid_amount are USD amounts at the service-line grain. billed_amount is the total for the line and must not be multiplied by units unless a separately named per-unit field is supplied by an authoritative source. A numeric 0.00 is a valid value where allowed by the schema and is not equivalent to null. service_date is the as-of date used to evaluate member eligibility, plan coverage, and the applicable policy version.

Source of truth, freshness & versioning: within this microcosm, the immutable raw claim_line_items.csv record is the canonical source for line-level procedure, service date, units, and amounts; claims_ledger.csv is canonical for claim-header totals. The AI-ready layer should use the latest validated ingestion batch available at decision time and retain source-file/batch provenance. Semantic contract claim_line_items_semantic_contract_v1.0 is aligned to schema v1.0; any breaking change to grain, field meaning, units, or as-of semantics requires a new contract version.

Consumer interpretation to preserve: the AI-ready transformation may enrich a line with validated context, but it must preserve claim-line grain, distinguish zero from null, treat billed_amount as the line total, and perform temporal joins using service_date. Any correction to a source value must be explicit, traceable, and flagged rather than silently changing business meaning.

AI Context / Quality Controls

Rule / Expectation	What Goes Wrong	What AI Receives	Possible AI Impact	Control / Response
Claim-line grain: one row per service line; claim_line_id is unique and (claim_id, line_number) is unique within a claim.	Duplicate rows or many-to-many joins change the declared grain.	Repeated service lines and repeated claim context that look individually valid.	Inflated totals, duplicated evidence, or distorted utilization.	Validate PK/FK and (claim_id, line_number) uniqueness; assert join cardinality; quarantine duplicates before the AI-ready layer.
Amount semantics: billed_amount is the total USD amount for the service line, not a per-unit amount.	An upstream source changes the unit meaning but keeps the same numeric field and type.	A schema-valid amount with the wrong business unit/meaning.	Line cost can be under- or overstated, affecting summaries and review signals.	Enforce the semantic contract and unit metadata; normalize before AI; never derive or reinterpret per-unit values silently; flag contract mismatches.
Point-in-time context: service_date drives eligibility, plan, and policy selection.	A valid date is wrong for the service, or a current plan/policy is used instead of the as-of version.	A schema-valid line paired with the wrong coverage or policy context.	Incorrect approve/deny/escalate recommendation.	Validate service_date against the parent claim window; perform temporal joins on effective/termination dates; preserve policy version/jurisdiction; flag no-match or multi-match cases.

Supporting evidence: metadata/structured_data_dictionary.csv; metadata/schema_claim_line_items_v1.csv; metadata/structured_to_knowledge_linkages.csv; data/structured/raw/claim_line_items.csv.

4. Governance, Metadata & AI Lineage

Anchor dataset: *insurance_plans.csv* — plan PLAN-001 (Silver HMO), linked to policy HLT-POL-002 (QI-002).

Data Owner & Source of Record. Owner: Payer Operations. System of record: **Core Plan Administration System** — the [insurance_plans](#) extract is derived, not authoritative.

Classification & Access. *insurance_plans* is Internal/Confidential; *patient_accounts/claims_ledger* are Restricted (PHI). Joins require RBAC + audit logging; AI gets read-only, minimum-necessary fields.

Freshness & Point-in-Time. Terms are valid only within *effective_date–termination_date*; AI must match plan version to claim service date, not the current row — the gap **QI-002** exposes. SLA: ≤24h stale after plan updates.

Contextual Metadata. Every record carries *plan_id* + effective window, *authority_status* (guards **QI-001**'s draft doc), *coverage_region*, and *source_system*/timestamp. See [structured to knowledge linkages.csv](#).

Lineage Trace. **Source** (Plan Admin. System) → **Transformation** (validate, tag *authority_status*, filter to service date) → **AI-ready Representation** (point-in-time plan + policy) → **AI Context** (correct benefit terms for reviewer).



Result: the AI gets the correct **meaning, timing, classification,** and **traceability** to defend a claim decision back to its source.